

Master DevOps
with Vinay

Jenkins

Build Failures and Solutions

DAY 3



MASTER DEVOPS WITH VINAY

1. Authentication Failures in Jenkins Pipelines

Problem:

Pipelines fail when attempting to access external systems like Git repositories, Docker registries, or cloud services due to missing or invalid credentials.

Solution:

Ensure secure and reliable access by following these best practices:

Store Credentials Securely in Jenkins

Navigate to:

Manage Jenkins → Credentials

Add and manage credentials securely using appropriate scopes (e.g., Global or Folder-specific).

Reference Credentials in Pipelines Properly

Use the `credentialsId` parameter in relevant steps. Example for Git:

```
git credentialsId: 'my-credentials-id', url: 'https://github.com/your-repo.git'
```

Manually Test Credentials Before Pipeline Use

Verify credentials by manually cloning repos, logging into registries, or connecting to external services to catch issues early.

 **Pro Tip:** Use Jenkins credential bindings to inject secrets as environment variables without exposing them in logs.

2. SCM Checkout Errors in Jenkins Pipelines

Problem:

Pipelines may fail during source code checkout due to issues like incorrect repository URLs, invalid branch names, or misconfigured credentials.

Solution:

 Double-check the Git repository URL and branch name in your pipeline script to ensure accuracy.

 Use the Jenkins Pipeline Syntax Generator to build the correct SCM checkout steps tailored to your use case.

 For declarative pipelines, use:

```
checkout scm
```

Or for more control:

```
git branch: 'main',  
  url: 'https://github.com/my-repo.git',  
  credentialsId: 'my-credentials-id'
```

 **Pro Tip:** Ensure that the Jenkins credentials used have the appropriate read access to the Git repository and that the branch exists and is not protected from CI tools.

3. Stale Workspace Issues in Jenkins Pipelines

Problem:

Pipelines may fail or behave unpredictably due to leftover files, cache, or artifacts from previous builds that persist in the workspace.

Solution:

- ✓ Clean the workspace at the start of each build using the built-in step:

```
cleanWs()
```

- ✓ Alternatively, enable this option directly in the job configuration:
"Delete workspace before build starts"

- ✓ Regular workspace cleanup ensures consistent, repeatable builds and helps prevent unexpected file conflicts or version mismatches.

 **Pro Tip:** Combine `cleanWs()` with artifact archiving logic to avoid losing valuable data before cleanup.

4. Dependency Management Failures in Jenkins Pipelines

Problem:

Build tools such as Maven, Gradle, or npm fail to resolve required dependencies, causing the pipeline to break.

Solution:

- ✓ Check network access to external or internal artifact repositories (e.g., Maven Central, npm registry, Artifactory, Nexus).

- ✓ Enable caching to speed up builds and reduce external dependency issues:
 - Use caching solutions like Artifactory or Nexus
 - Integrate caching into your pipeline for consistency

- ✓ Avoid hardcoded repository URLs, instead, use environment-specific config files to support different build environments (e.g., dev, staging, prod).

 **Pro Tip:** Always version-lock dependencies where possible to prevent unexpected breakages due to upstream changes.

5. Credential Management Challenges in Jenkins Pipelines

Problem:

Sensitive credentials are sometimes exposed in logs or hardcoded, leading to potential security risks.

Solution:

- ✓ Store all secrets securely in Jenkins:

Navigate to:

Manage Jenkins → Credentials

Use folders and credential types wisely (username/password, secret text, etc.).

- ✓ Inject credentials securely into your pipeline using `withCredentials`:

```
withCredentials([string(credentialsId: 'my-secret', variable: 'SECRET')]) {
  sh 'do-something-secure.sh'
}
```

- ✓ Avoid logging secrets:
Never use `echo` or `print` statements to display sensitive values.

 **Pro Tip:** Periodically rotate credentials and audit access to Jenkins credentials for enhanced security posture.

6. Parallel Stage Failures in Jenkins Pipelines

Problem:

Pipelines may fail when parallel stages conflict, typically due to shared resources, workspace collisions, or dependency overlaps.

Solution:

- ✓ Design parallel stages to be fully independent. Avoid shared directories, files, or tools that can create contention.
- ✓ Use separate workspace directories to isolate tasks:

```
parallel {
  stage('Test') {
    steps {
      script {
        dir('test-workspace') {
          sh 'run-tests.sh'
        }
      }
    }
  }
  stage('Build') {
    steps {
      script {
        dir('build-workspace') {
          sh 'build.sh'
        }
      }
    }
  }
}
```

- ✓ Prevent race conditions by applying locks on shared resources:

```
lock('shared-resource') {
  sh 'critical-operation.sh'
}
```

 **Pro Tip:** Name locks descriptively and monitor build concurrency to maintain high pipeline reliability.

7. Pipeline Aborted by User

Problem:

When a pipeline is manually stopped, it often leaves behind temporary files, containers, or VMs, consuming unnecessary resources.

Solution:

- ✓ Use a try-catch-finally block to ensure cleanup always runs—even if the build is interrupted:

```
try {
  sh 'run-critical-task.sh'
} catch (Exception e) {
  echo "Error occurred: ${e}"
} finally {
  sh 'cleanup.sh'
}
```

- ✓ Place all teardown logic in the finally block to guarantee execution regardless of failure or manual termination.

 **Pro Tip:** Combine with post blocks for even more reliable cleanup across all stages.

8. Disk Space Issues in Jenkins Pipelines

Problem:

Builds fail due to insufficient disk space on Jenkins master or agent nodes, often caused by accumulated artifacts, logs, and workspaces.

Solution:

- ✓ Configure automatic cleanup of old builds and artifacts:
 - Go to: Manage Jenkins → Configure System → Workspace Cleanup
 - Set retention policies based on build age or count.
- ✓ Use the `cleanWs()` step to clear workspaces after builds:

```
post {
  always {
    cleanWs()
  }
}
```

- ✓ Monitor disk usage regularly and set up alerts to catch low space conditions early, use tools like `df`, `du`, or Jenkins monitoring plugins.

 **Pro Tip:** Store large artifacts externally (e.g., in Artifactory or S3) and limit what gets archived in Jenkins.

9. Docker Integration Issues in Jenkins Pipelines

Problem:

Docker-related pipeline steps may fail due to missing permissions, Docker daemon errors, or network issues on Jenkins agents.

Solution:

- ✓ Verify Docker installation and access on all Jenkins agents:
 - Ensure Docker is installed and the agent user belongs to the `docker` group.
 - Test with: `docker info` or `docker ps` on the agent.
- ✓ Use Docker agents correctly in declarative pipelines:

```
pipeline {
  agent {
    docker {
      image 'node:14'
    }
  }
  stages {
    stage('Build & Test') {
      steps {
        sh 'npm install && npm test'
      }
    }
  }
}
```

- ✓ Manually test Docker commands on agents to validate configuration before relying on them in production pipelines.

 **Pro Tip:** Use Docker-in-Docker (DinD) with caution—ensure it's securely configured to avoid privilege escalation risks.

10. SCM Polling Failures in Jenkins

Problem:

Pipelines fail to trigger because changes in source control aren't detected, resulting in missed builds.

Solution:

- ✓ Enable SCM Polling under the Build Triggers section of your job configuration.
- ✓ Set an appropriate polling schedule using cron syntax:

```
triggers {  
  pollSCM('H/5 * * * *') // Polls every 5 minutes  
}
```

Recommended:

Switch to webhooks (e.g., from GitHub, GitLab, Bitbucket) for faster, event-driven triggers and reduced server load.

Pro Tip:

Use polling only as a fallback or for systems where webhooks are not supported.